AI Debate & Interview Coach

This project is a full-stack AI mock interview platform for resume-based interview practice. It helps a candidate upload a resume, start a live interview, answer questions verbally, and receive feedback on voice performance, answer quality, and overall interview readiness.

1. Project Overview

The application combines:

- AI-powered interview generation based on a candidate resume
- Live voice capture through browser speech recognition
- Optional camera-based face and expression monitoring
- Final score and performance report generation
- A FastAPI backend for interview orchestration and evaluation
- A Next.js frontend for the user experience

This is designed to simulate a realistic hiring interview flow rather than a static list of canned questions.

---

2. What the App Does

Core user flow

1. User uploads their resume in PDF, DOCX, or TXT format.
2. Backend extracts the resume text and sends it to the LLM.
3. The AI asks the first interview question based on the candidate background.
4. The candidate answers aloud using the browser microphone.
5. Speech is converted to text and processed by the AI.
6. The interviewer asks the next relevant question based on the resume and topic coverage.
7. After the interview, a final summary is generated with:
    - overall score
    - strong areas
    - weak areas
    - question-by-question assessment
    - suggestions for improvement

Extra features

- AI voice playback for interviewer prompts
- Live silence detection and auto-turn progression
- Hands-free interview mode
- Face monitoring for eye contact and expression quality
- Resume-grounded question flow instead of generic Q&A

---

3. Tech Stack and Purpose

Layer	Technology	Why it is used here
Frontend app shell	Next.js	Main UI framework for the web app and routing
Frontend UI	React + TypeScript	Component-based interface and safer typed frontend code
Styling	Tailwind CSS	Fast UI design for interview dashboard, cards, status states
AI interview backbone	Groq LLM API	Generates resume-based questions and evaluates answer quality
Backend API	FastAPI	High-performance Python API layer for interview endpoints
Resume parsing	PyPDF + zip/XML parsing	Extract text from PDF and DOCX resumes
File upload handling	python-multipart	Accept resume files in the backend
Speech recognition	Web Speech API (SpeechRecognition)	Converts user voice to text in the browser

AI voice playback	Web Speech API (SpeechSynthesis)	Reads interviewer prompts aloud to the user
Face analysis	face-api.js	Detects facial expressions, gaze, and camera-based attention
Charts and analytics	Recharts	Builds scorecards and visual report data
Environment configuration	python-dotenv	Loads backend environment values like API keys
Development server	Uvicorn	Runs the FastAPI backend locally
Package management	npm + pip	Installs frontend and backend dependencies

---

4. Architecture Overview

Frontend

Location: `frontend/`

Used for:

- resume upload page
- live interview UI
- microphone controls
- AI voice interaction
- scorecards and report panels
- camera + face tracking

Key frontend files:

- `frontend/app/mock-interview/page.tsx` — resume upload and interview entry page
- `frontend/app/components/ChatUI.tsx` — main interview experience, speech logic, score generation, AI conversation UI
- `frontend/app/dashboard/page.tsx` — dashboard view
- `frontend/app/practice/page.tsx` — practice session UI

Backend

Location: `backend/`

Used for:

- resume extraction
- interview question generation
- answer evaluation
- conversation orchestration
- summary report generation
- API endpoints for frontend requests

Key backend files:

- `backend/main.py` — app initialization and router registration
- `backend/routes/mock_interview.py` — interview lifecycle logic
- `backend/routes/analysis.py` — feedback and analysis endpoints
- `backend/routes/sessions.py` — session data management
- `backend/routes/debate.py` — debate-related logic

API flow

The frontend talks to the backend through proxy routes configured in `frontend/next.config.js`.

Examples:

- `/api/mock-interview/start-interview`
- `/api/mock-interview/chat`
- `/api/mock-interview/interview-summary`
- `/api/analyze`

This keeps the frontend and backend decoupled while letting the local app behave like a single system.

---

5. Project Folder Structure

```
ai-debate-coach/
├── backend/
│   ├── main.py
│   ├── requirements.txt
│   ├── routes/
│   │   ├── analysis.py
│   │   ├── debate.py
│   │   ├── mock_interview.py
│   │   ├── sessions.py
│   │   └── __init__.py
│   └── .env.local
├── frontend/
│   ├── app/
│   ├── public/
│   ├── package.json
│   ├── next.config.js
│   ├── tailwind.config.js
│   └── tsconfig.json
├── README.md
└── .gitignore
```

---

6. Main Technologies Used by Feature

Resume-based interview flow

- `FastAPI` for API endpoints
- `Groq` for LLM generation
- `PyPDF` for PDF resume parsing
- `zipfile` and XML parsing for DOCX resumes
- `python-multipart` to receive uploaded files

Voice interview behavior

- browser `SpeechRecognition` for live transcription
- browser `SpeechSynthesis` to speak questions back to the user
- `React state` and `useEffect` to control turn flow and auto-advance behavior

Candidate monitoring

- `face-api.js` for expression detection
- `video` capture from browser camera
- logic to detect attention state, emotion labels, and risk alerts

Reporting and evaluation

- `Groq` LLM to score answer quality and produce final interview insight
- `React` components to show strengths, gaps, and performance cards
- `Recharts` for analytics-style visualization in dashboard pages

---

7. Interview Logic

The app simulates a real-time hiring interview flow:

1. A resume is uploaded.
2. The backend extracts the resume text.
3. The app calls the AI to generate the first question.
4. The candidate answers via microphone.

5. The AI evaluates the answer and asks a new question.
6. A final summary is generated after the interview is complete.

This is done to behave more like an interview coach than a static quiz engine.

---

8. Setup Instructions

Backend

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Create `.env.local` in the backend folder with:

```
GROQ_API_KEY=your_key_here
GROQ_MODEL=openai/gpt-oss-120b
```

Run backend:

```
cd backend
source .venv/bin/activate
python main.py
```

The backend should run on:

- `http://127.0.0.1:8000`

Frontend

```
cd frontend
npm install
npm run dev -- --hostname 0.0.0.0 --port 3001
```

Open:

- `http://127.0.0.1:3001`

---

9. Browser Requirements

For the speech and face features to work correctly, use:

- Chrome
- Edge
- Safari (with microphone support)

You must allow:

- microphone permission
- camera permission if using proctoring mode

Speech recognition is browser-dependent and may require user interaction before it starts reliably.

---

10. Best Explanation of the Stack in One Sentence

This project uses `Next.js + React + TypeScript` on the frontend for the interview UI, `FastAPI + Python` on the backend for orchestration and resume processing, `Groq` for AI question generation and evaluation, `Web Speech API` for live voice interaction, and `face-api.js` for camera-based interview monitoring.

---

11. Summary

This is a real-world AI interview coaching app that combines:

- resume parsing
- LLM interview generation
- voice input/output
- facial monitoring
- final interview scoring

It is not just a frontend page; it is a complete AI-powered mock interview system with backend intelligence and browser-based interaction.